

Secrets revealed in this session:

**To explore the principles of
quantum machine learning
models, their parameterisation
and optimisation**



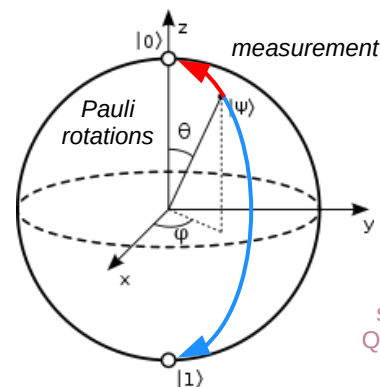
QML workshop
QML team
QML and aims
Parameterised circuits
Data encoding
Angle encoding
The good, the bad and the ugly
State measurement
Quantum model training
Parameters optimisation
Model geometry and gradients
QML readings
PennyLane demo
Summary

Quantum Machine Learning

Introduction

Jacob L. Cybulski

Enquanted, Melbourne, Australia



We will assume
some knowledge of
Quantum Computing
ML and Python



Jacob Cybulski

Founder, Researcher, Consultant
Enquanted, Australia

and
Honorary A/Prof
In Quantum Computing
Deakin University
[LinkedIn](#)



Dr Paweł Gora

Coordinator of QPoland *and*
CEO of Fundacja Quantum AI
[LinkedIn](#)

In Association with:

- QPoland
- QSaudiArabia
- Fundacja Quantum AI

Mentors:

- Tomasz Rybotycki
- Sebastian Zajac
- Paweł Gora
- Ola Tarig
- Mohamedelfateh Seedahmed
- Khulud Almutairi
- Abrar

Quantum Machine Learning

vs. Quantum Data Science vs. Quantum AI

Quantum Machine Learning: *methods+tools*

is a field of study applying methods and theories of quantum mechanics,

to redefine machine learning concepts and techniques,

for developing and applying new *learning tools, algorithms and models*

Quantum Data Science: *knowledge*

is a field using *scientific methods* + principles of quantum mechanics,

to create new approaches to extracting and analysing insights, patterns, and meaning from data,

for problem solving and decision-making

Quantum AI:

applications

is a field focusing on applying quantum computing,

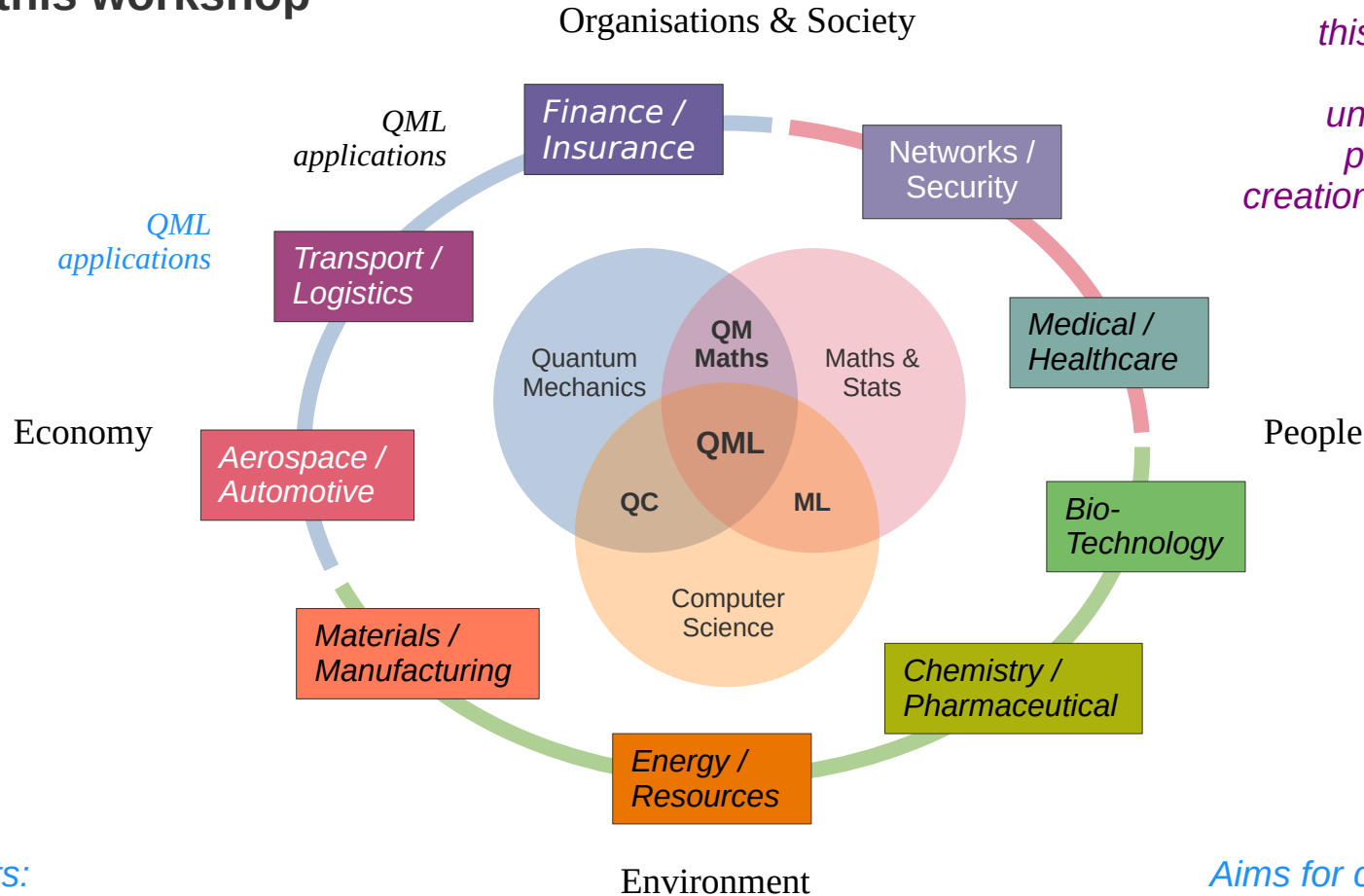
to develop new highly effective technologies,

for performing tasks attributed to human cognition, with *effectiveness approaching human intelligence*

QML

Quantum ML

aims of this workshop



this workshop aims at developing the understanding of and practical skills in the creation and application of QML models

*Aims for physicists:
Learn about ML in QML*

*Aims for computer scientists:
Learn about Q in QML*

ML/QML objective and categories

Objective	Category	Core Goal	Common Quantum Counterpart (QML)
Classification	Supervised	Assign a discrete label or category	Quantum Support Vector Machines (QSVM)
Estimation	Supervised	Predict a continuous numerical value	Variational Quantum Regressors (VQR)
Clustering	Unsupervised	Find hidden groupings in data	Quantum K-Means / Quantum Clustering
Association	Unsupervised	Discover rules and relationships	Quantum Association Rule Mining
Dimension Reduction	Unsupervised	Compress data while retaining essence	Quantum PCA / AE (QPCA / QAE)
Ranking	Hybrid/Supervised	Determine relative order or relevance	Quantum-enhanced Recommendation
Anomaly Detection	Unsupervised	Identify outliers or abnormal data	Quantum Kernel Anomaly Detection
Synthesis / Generation	Generative	Create new, realistic synthetic data	Quantum Circuit Born Machines (QCBM)
Transformation / Mapping	Reconstruction	Map data from one form/state to another	Quantum Autoencoders (QAE / DQAE)
Action Optimization	Reinforcement	Learn a policy to maximize rewards	Quantum Reinforcement Learning (QRL)

In this workshop, we will only discuss quantum classifiers, estimators and autoencoders.

QML platforms / SDKs

Qiskit, PennyLane, Cirq, Yao, ...

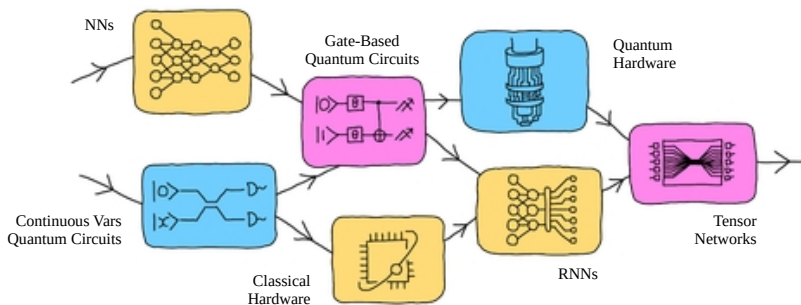
- **Qiskit** (Application driven $\approx$ high-level APIs)
 - **Location:** USA
 - **Language:** Python, Rust (core)
 - **Company:** IBM Research
 - **Backends:** IBM, AQT, IQM, Rigetti, Quantinuum
 - **Models:** VQC, VQR, QNN, QCNN, QSVM, QGAN, Q Kernels, VQE, VQLS, QFT, QAOA
 - **ML SDK:** Scipy, PyTorch, Scikit-Learn
 - **Apps:** QML, Finance, Optimization, Nature
- **Cirq** (Hardware centric $\approx$ low-level APIs)
 - **Location:** USA
 - **Language:** Python (core)
 - **Company:** Google Quantum AI
 - **Backends:** Google, AQT, IonQ, Pasqal, Rigetti
 - **Models:** VQE, QAOA, via TFQ also QNN, QCNN, QRNN, QGNN, QGAN, QRL, Q kernels, etc.
 - **ML SDK:** Tensorflow Quantum (TFQ)
 - **Apps:** QML, Chem, Materials, Comms, Metrology
- **Other Platforms / Q-SDKs**
Classiq / Classiq, Forest / Rigetti, Ocean / D-Wave, Quantum Development Kit with Q# / Microsoft, cuQuantum / Nvidia, t|ket> / CQC
- **PennyLane** (Diff-programming $\approx$ functional APIs)
 - **Location:** Canada
 - **Language:** Python, C++ (lightning modules)
 - **Company:** Xanadu
 - **Backends:** Xanadu, AQT, IonQ, Rigetti, Honeywell
 - **Models:** QNN, Q Kernels, QFT, QAOA, via PyTorch also CNN, RNN, LSTM, GRU, LLMS, GAN
 - **ML SDK:** PyTorch, Scipy, Scikit-Learn (+JAX, Numpy)
 - **Apps:** QML. Optimization, Chemistry
- **Yao** (Array theoretic $\approx$ matrix-block APIs)
 - **Location:** China / Taiwan
 - **Language:** Julia (core)
 - **Company:** QuantumBFS
 - **Backends:** Simulators, acceleration via Nvidia GPUs
 - **Models:** VQE, many others via Julia (Flux)
 - **ML SDK:** via Julia/Python (scipy, sklearn)
 - **Apps:** Via Julia (QML, AI, Optimization, Physics, Chemistry, Biology, Earth, Finance, Robotics)

QML with PennyLane

Everything is a function!



- Supports *differentiable programming paradigm*
- Supports *hybrid quantum-classical models*



- Integrates with *Python*
- Executes on *CPUs*, *GPUs* and *QPUS*
- It is quantum device independent
- Supports *error mitigation*
- Extends its selection of *gradients* with those from JAX, PyTorch, TensorFlow, or NumPy
- Provides numerous quantum models, such as: *QNNs*, *quantum kernels* and *Fourier models*
- Can be extended with *PyTorch* models / optimisers.

Make your hands dirty with PennyLane:

- Create a simple PL model to fit a simple function
- Learn to initialise model weights
- Explore the impact of model structure on performance
- Create minimalistic quantum models
- Learn the interaction of data encoding and ansatz
- Investigate different types of entangling
- Apply the best solution to more complex data
- Develop stamina for QML development

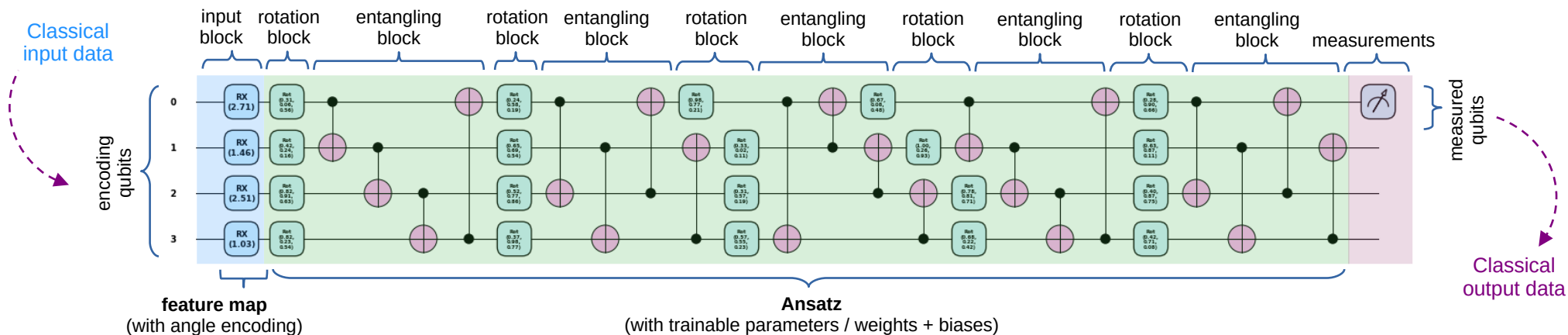
Be ready for the key QML takeaways:

- Plan model development, tests and experiments
- Bad data encoding spoils the bunch!
- Strong entanglement improves the data fit
- More width and depth = the curse of dimensionality
- Carefully consider your quantum model initialisation
- Surprise - a single qubit model still works! (and well)
- More training does not solve the problems
- Data reuploading makes a huge difference!

Variational Quantum Models

= Parameterised Quantum Circuits

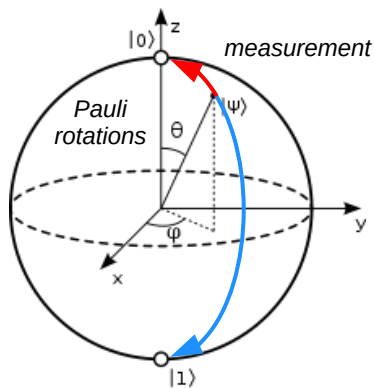
Variational quantum circuits are not executable!
They must first be instantiated, i.e. all of their
input and weight parameters must be assigned values!
Ansatz parameters are trainable.



We can create a “variational” model = a circuit template with parameterised gates, e.g. $P(a)$, $R_y(a)$ or $R_z(a)$, each allowing rotation of a qubit state in x, y or z axis (as per Bloch sphere).

Typically, but now always, the circuit consists of three blocks:

- a feature map (input)
- an ansatz (processing)
- measurements (output)



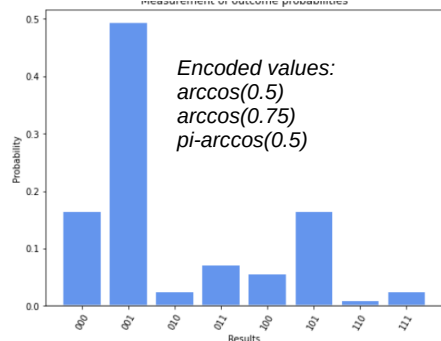
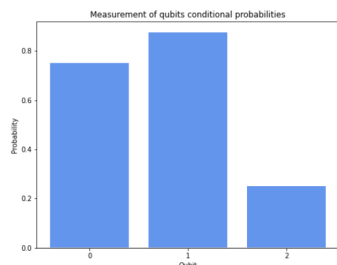
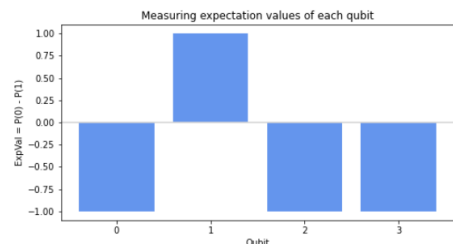
Classical input data is encoded into the feature map's parameters, setting the model's initial quantum state.

The quantum state is then altered by an ansatz, which consists of parameterised gates (operations), which alter the circuit state. Ansatz parameters are trainable. Qubits and parameters increase the model dimensionality.

The quantum state of the circuit is then measured and interpreted as the model's output in classical data form, e.g. as binary values, integer or real value, a single event's probability or the probability distribution.

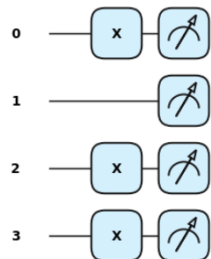
Data Encoding

Many encoding methods, e.g. basis, angle, amplitude, QRAM, ...



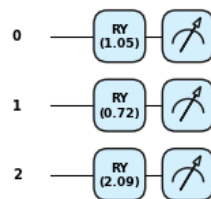
Encoding can be repeated across the circuit, which is called data reuploading

Basis encoding



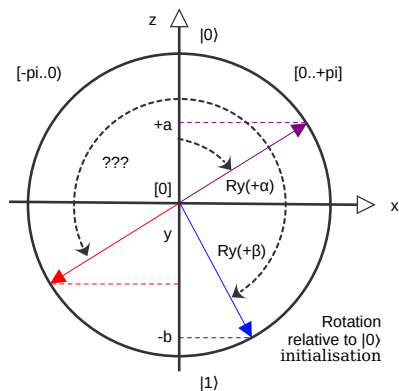
The simplest data encoding and very popular, however, little data can be encoded at a time, you can encode only binary values per each qubit.

Angle encoding



One of the most flexible and very effective, you can encode floating point numbers.

What you encode depends on your intention!



Basis encoding + Expectation measurement (local cost)

```
@qml.qnode(dev)
def basis_circuit_2A(features):
    for i in range(len(features)):
        if features[i] == 1: qml.PauliX(i)
    return [qml.expval(qml.PauliZ(m)) for m in range(len(features))]
```

Angle encoding + Probability measurement (local cost)

```
@qml.qnode(dev)
def angle_circuit_1b(features):
    qml.AngleEmbedding(features=features, rotation='Y',
                        wires=range(len(features)))
    return [qml.probs(m) for m in range(len(features))]
```

Angle encoding + Probability measurement (global cost)

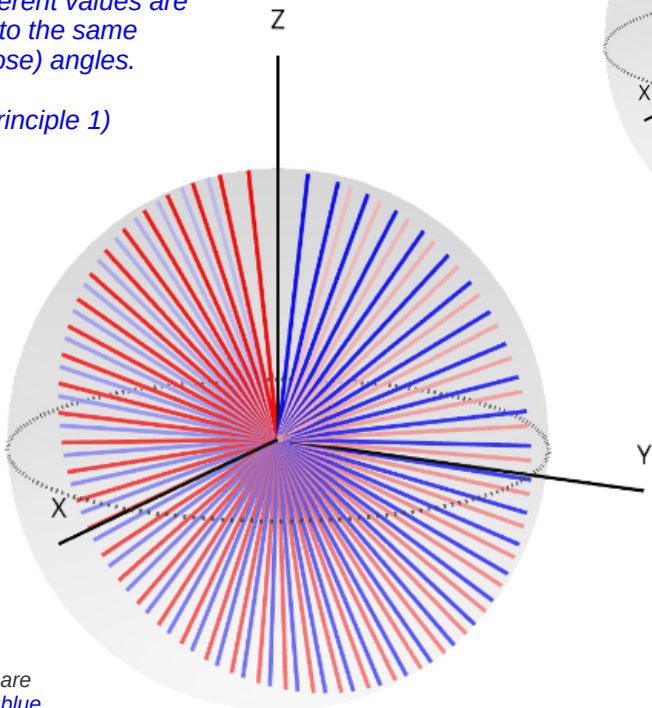
```
@qml.qnode(dev)
def angle_circuit_1a(features):
    qml.AngleEmbedding(features=features, rotation='Y',
                        wires=range(len(features)))
    return qml.probs(range(len(features)))
```

Angle encoding

The Good, the Bad and the Ugly

This example shows encoding values wrapping around the Bloch sphere (e.g. -6 to 6), so that different values are mapped into the same (or very close) angles.

(violates principle 1)

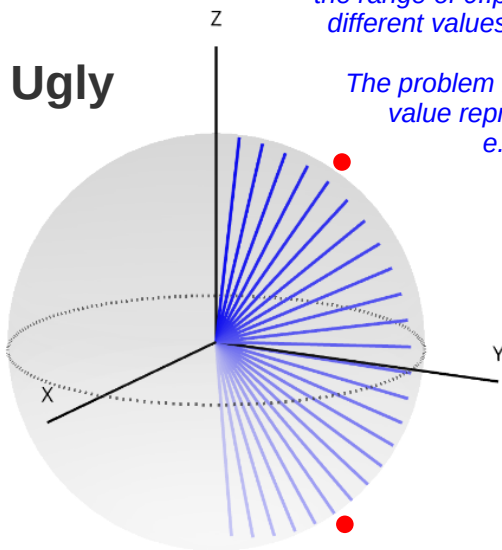


Positive values are represented by blue vectors and negative values by red vectors.

This example shows encoding values wrapping around the range of $0..pi$ of the Bloch sphere, ensuring that different values are represented by unique angles.

The problem may arise if we may have the same value represented by two distinct amplitudes, e.g. values of $\sin(x)$ represented by x

(violates principle 2)



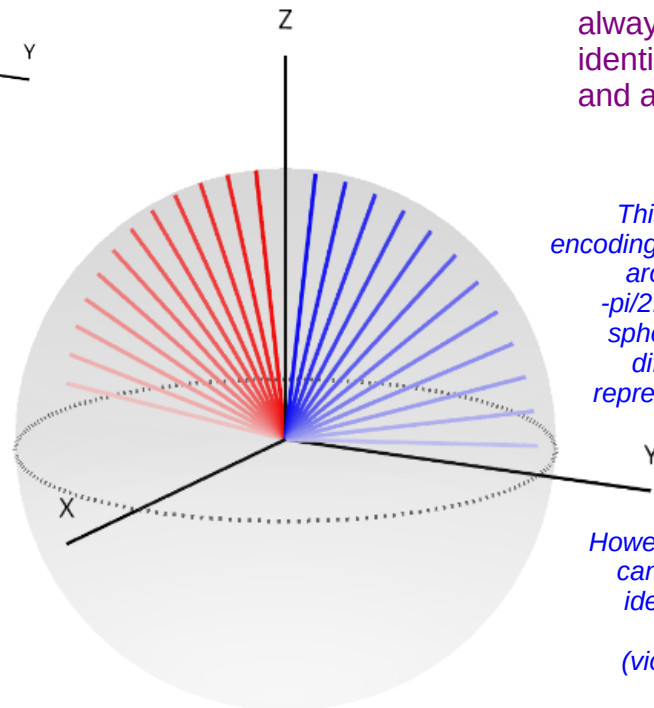
Special care must be taken in those cases where the values form some kind of symmetry or repetition. e.g. in a case values x and $2pi+x$ effectively represent the same angular position and their encoding should also be identical.

The red dot represents the same value which on two occasions maps onto two different angles and thus two amplitudes.

Two principles of quantum data encoding:

- 1) distinct data values should map onto distinct angles and amplitudes
- 2) the same data values should always map into identical angles and amplitudes

This example shows encoding values wrapping around the range of $-pi/2..pi/2$ of the Bloch sphere, ensuring that different values are represented by unique angles.



However, some angles can be measured as identical amplitudes

(violates principle 1)

```
@qml.qnode(dev)
def basis_circuit_2B(features):
    for i in range(len(features)):
        if features[i] == 1: qml.PauliX(i)
    qml.Barrier()
    for i in range(len(features)-1):
        qml.CNOT([i, i+1])
    qml.Barrier()
    return qml.expval(qml.PauliZ(3)) # Only 1 value returned
```

local
expectations
measurement

```
@qml.qnode(dev)
def basis_circuit_2A(features):
    for i in range(len(features)):
        if features[i] == 1: qml.PauliX(i)
    return [qml.expval(qml.PauliZ(m)) for m in range(len(features))]
```

local
expectations
measurement

```
@qml.qnode(dev)
def basis_circuit_2C(features):
    for i in range(len(features)):
        if features[i] == 1: qml.PauliX(i)
    qml.Barrier()
    for i in range(len(features)-1):
        qml.CNOT([i, i+1])
    qml.Barrier()
    tensor_product = qml.prod(*(qml.PauliZ(i) for i in range(n_wires)))
    return qml.expval(tensor_product) # Only 1 value returned
```

global
expectations
measurement

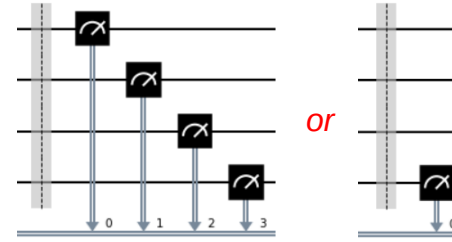
```
@qml.qnode(dev)
def angle_circuit_1a(features):
    qml.AngleEmbedding(features=features, rotation='Y',
                        wires=range(len(features)))
    return qml.probs(range(len(features)))
```

global probability
measurement

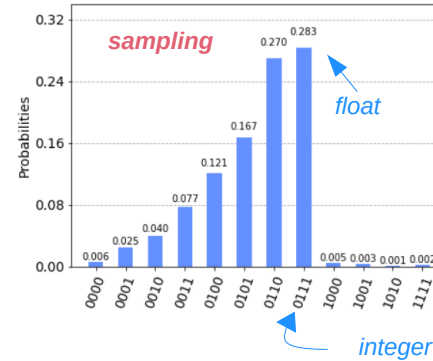
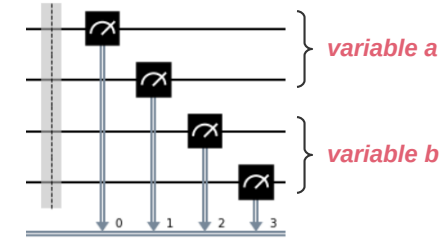
global cost =
exponential outcome

local cost =
linear outcome

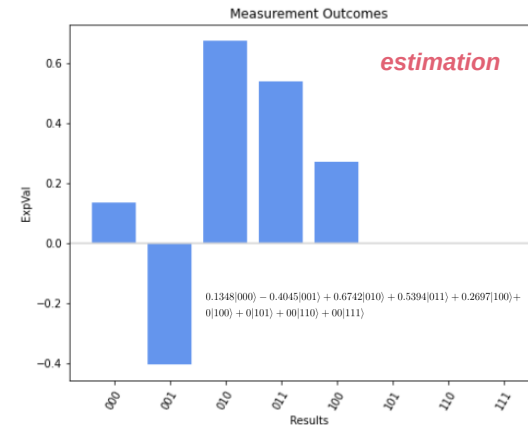
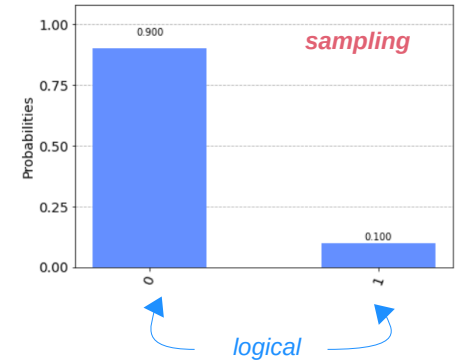
This is simplistic, as the distinction between local and global cost
formally is linked to flattening of gradients



OR



OR



Measurements must be
repeated, collected and
then can be interpreted in
many different ways

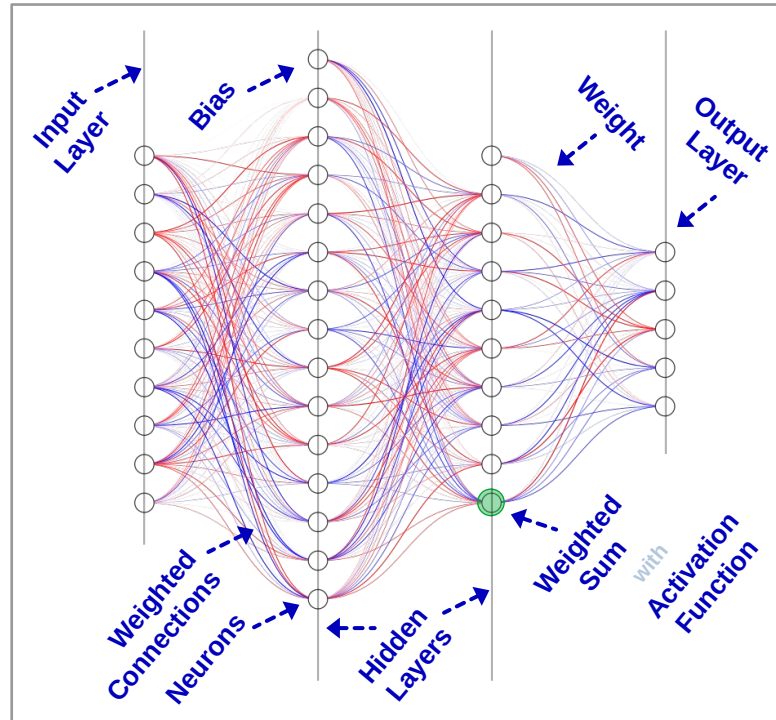
It is also possible to
measure mid-circuit,
however, beware as the
circuit is no longer unitary
and not reversible!

State Measurement

Training of classical Neural Networks

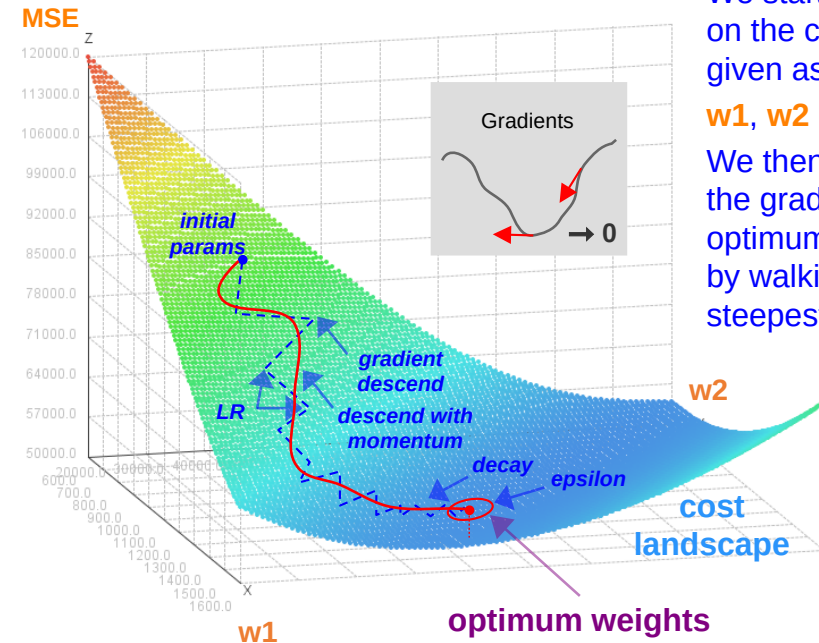
Multilayer Perceptron (MLP)

Training of hybrid
quantum-classical models
is almost identical



- MLP is capable of learning any “smooth” function by associating inputs with outputs
- Other NNs: CNN, AE, GAN, LSTM

Now we will look for the best NN model, i.e. the model weights which generate the model of the lowest cost (e.g. MAE)



optimum weights
at $\min(\text{MSE}) \equiv$
 $\text{gradient} \rightarrow 0$

Gradient descent:

We start at any point on the cost landscape given as:

$w_1, w_2 \dots$ and MSE

We then search for the $\text{gradient}=0$, i.e. optimum weights, by walking down the steepest slope

Model design & training

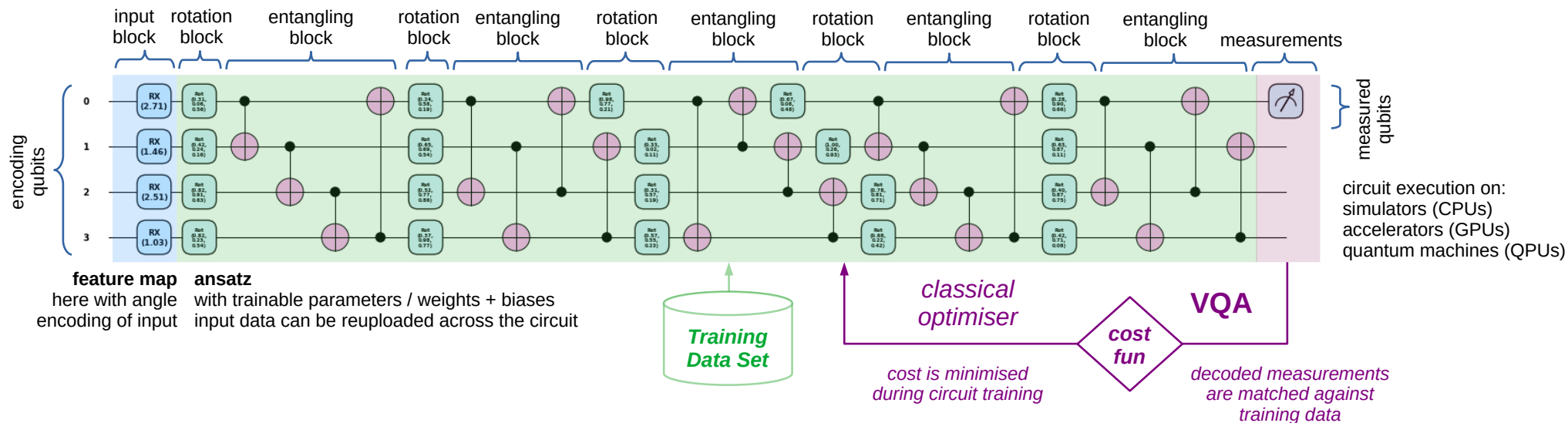
A simple quantum classifier ...

Beware that
adding qubits adds
parameters and entanglements!

The number of states represented by the
circuit **grows exponentially** with the
number of qubits!

Beware that
adding 1 measurement
doubles the number of outcomes!

So... having n
measurements leads to
 2^n outcomes



feature maps vary in:
structure and function

- encoding type
 - phase utilisation
 - repetition
 - distribution
- interwoven with
ansatz layers*

ansatz may vary in:

- width (qubits #)
- depth (layers #)
- dimensions (param #)
- structure (e.g. funnelling)
- entangling (circular, linear, sca)

ansatz layers consist of:

rotation blocks and entangling blocks
of R(x, y, z) and CNOT gates

(rotation) (entanglement)

different cost functions:

R2, MAE, MSE, Huber, Poisson, cross-entropy, hinge-embedding, Kullback-Leibner divergence

different optimisers:

gradient based (Adam, NAdam and SPSA)
linear approximation methods (COBYLA)
non-linear approximation methods (BFGS)
quantum natural gradient optimiser (QNG)

Creating your own ansatz

- Creating your own ansatz is technically simple but conceptually very challenging.
- An ansatz is the heart of a quantum model as it is responsible for the quantum model:
 - *Expressivity*, e.g. via the model overall architecture
 - *State evolution*, e.g. via the ansatz gates and layers
 - *Trainability*, e.g. via the ansatz parameters
 - *State space geometry*, e.g. via the ansatz entanglements
 - *Non-linear transformations*, e.g. via data reuploading
- Here, we focus on the “engineering” of an ansatz in PennyLane. Therefore, the ansatz needs to support:
 - *Flexibility* – it should allow variations, e.g. of its width and depth, type of entanglement and shape of weight params
 - *Modularity* – it should be part of a modular structure encapsulating and hiding implementation details, e.g. the shape and use of its weight and input parameters
 - *Reusability* – whenever appropriate it should rely on the API functions rather than re-implementing them, you should also be able to use the ansatz in many different contexts
 - *Hilbert Space* – ansatz main responsibility!

A PennyLane quantum model is a “circuit” function taking very specific parameters, commonly input and weights tensors only.

However, such a function needs to know much more to effectively process these data, e.g. the number of qubits, type of entanglement, etc.

For this reason, the function is created as **closure** generated by the function **factory**, which provides the closure with the **data environment** of values available at its creation, e.g. the circuit architectural details.

To be executable on any quantum device (including a CPU or GPU), first the closure needs to become a quantum node (QNode).

quantum model

```
### Define a quantum linear model functionally
### This is the PennyLane style of doing things
def qlinear(n_wires, scaler=1.0, basic=False):

    def _qlinear(weights, inputs):
        nonlocal n_wires, scaler, basic
        data_wires = list(range(n_wires))
        scaled_inputs = inputs * scaler

        # Standard angle encoding block
        qml.AngleEmbedding(scaled_inputs, wires=data_wires)

        # Ansatz with optional basic or strong entangling block
        if basic:
            qml.BasicEntanglerLayers(weights, wires=data_wires, rotation=qml.RY)
        else:
            qml.StronglyEntanglingLayers(weights, wires=data_wires)
        return qml.expval(qml.PauliZ(0))

    return _qlinear
```

function factory (with vars environment)

closure function (with nonlocal vars)

API calls to create gates with input and weight parameters

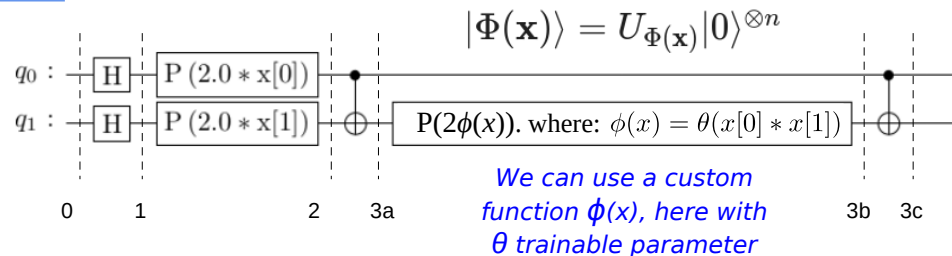
returns closure (with standard parameters)

matching model shape

```
### Check the model shape
### This is defined so that we could select and check the shape in one place
def qlinear_shape(n_wires, n_layers=1, basic=False):
    if basic:
        shape = qml.BasicEntanglerLayers.shape(n_layers=n_layers, n_wires=n_wires)
    else:
        shape = qml.StronglyEntanglingLayers.shape(n_layers=n_layers, n_wires=n_wires)
    return shape
```

What is the Hilbert space?

Example: Consider the following ZZ feature map



The story of the Hilbert space, the circuit states, the state vector paths and manifolds – and this is just the beginning...

the final state:

$$|\Phi(\mathbf{x})\rangle = \frac{1}{2} \begin{pmatrix} 1 \\ e^{i(2x_1+2\phi(x))} \\ e^{i(2x_0+2\phi(x))} \\ e^{i(2x_0+2x_1)} \end{pmatrix} \begin{matrix} \leftarrow |00\rangle \\ \leftarrow |01\rangle \\ \leftarrow |10\rangle \\ \leftarrow |11\rangle \end{matrix}$$

Step 0: Initial State

$$|\psi_0\rangle = |0\rangle \otimes |0\rangle = |00\rangle$$

Step 1: Superposition (Hadamard Gates $|+\rangle = \frac{|0\rangle+|1\rangle}{\sqrt{2}}$)

$$|\psi_1\rangle = (H \otimes H)|00\rangle = \frac{1}{2} (|00\rangle + |01\rangle + |10\rangle + |11\rangle)$$

Step 2: Phase Gates (adds phase when qubit is in state $|1\rangle$)

$$|\psi_2\rangle = \frac{1}{2} (|00\rangle + e^{i2x_1}|01\rangle + e^{i2x_0}|10\rangle + e^{i(2x_0+2x_1)}|11\rangle)$$

Step 3: Entanglement (CNOT – Phase – CNOT)

3a. First CNOT (Control q0, Target q1)

$$|\psi_{3a}\rangle = \frac{1}{2} (|00\rangle + e^{i2x_1}|01\rangle + e^{i2x_0}|11\rangle + e^{i(2x_0+2x_1)}|10\rangle)$$

3b. Middle Phase Gate $P(2\phi)$ on q1

$$|\psi_{3b}\rangle = \frac{1}{2} (|00\rangle + e^{i2x_1}e^{i2\phi}|01\rangle + e^{i2x_0}e^{i2\phi}|11\rangle + e^{i(2x_0+2x_1)}|10\rangle)$$

3c. Second CNOT (Control q0, Target q1)

$$|\psi_{3c}\rangle = \frac{1}{2} (|00\rangle + e^{i(2x_1+2\phi)}|01\rangle + e^{i(2x_0+2\phi)}|10\rangle + e^{i(2x_0+2x_1)}|11\rangle)$$

The Hilbert space is a coordinate system and a space of directions that the state of your model could take during its execution. As it executes it leaves a path, a “trail” its state vector follows. Because of entanglement, the state vector is a “complex” arrow and the trail becomes “knotted”. Trainable parameters allow adjusting the direction we take. By varying encoded input data and model parameters, all possible paths create a smooth curved surface – a manifold.

Parameter space

(dim = the number of params)
is a classical multi-dim space of trainable gate parameters, which the optimiser navigates.

Classical parameters

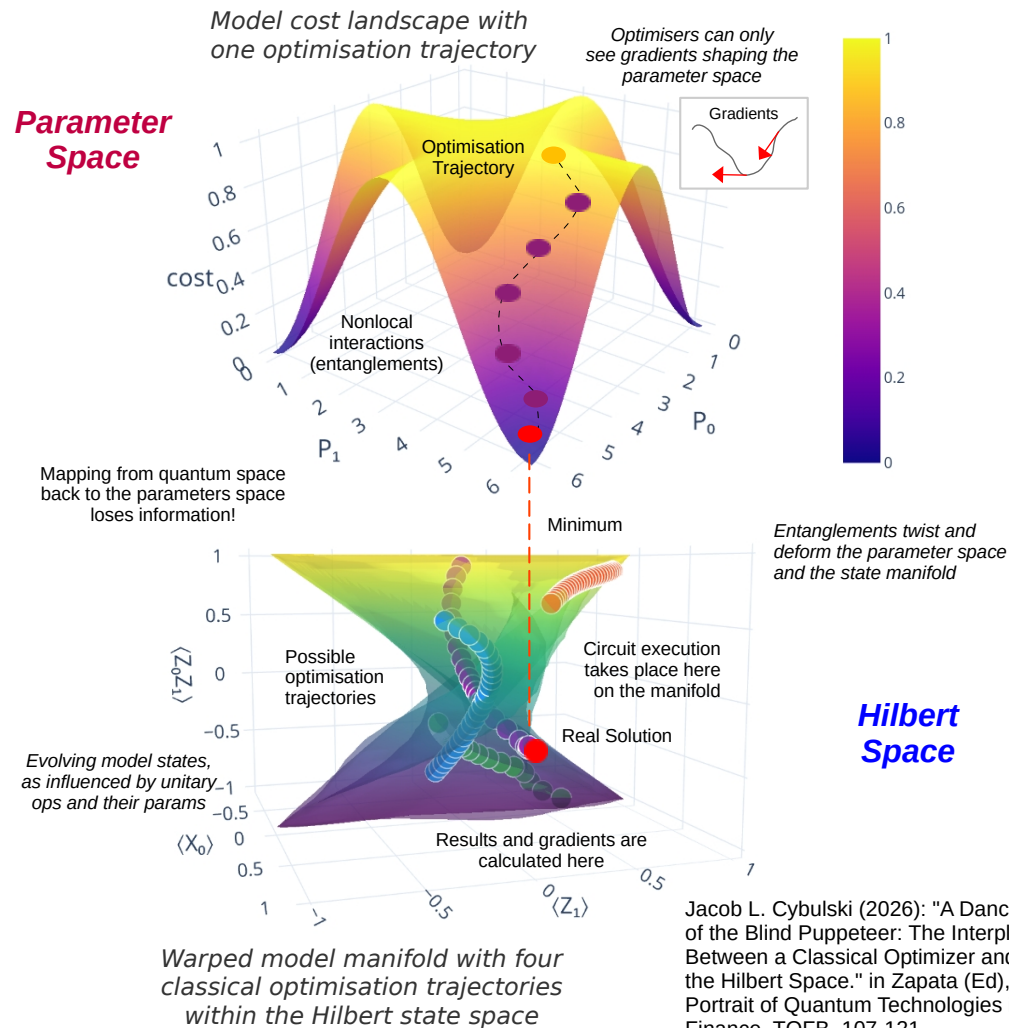
A classical optimiser can only manipulate the gate parameters but is not aware of their action on qubits states or their correlations via qubit entanglements.

Measurement

of individual qubits collapses their states, projecting the circuit state onto classical outcomes - the only information reaching the optimiser. In the process we lose some quantum information (e.g. phase).

Sebastian Zając, Jacob L. Cybulski, Bartosz Dziewit and Tomasz Kulpa (2026): "The Inverse Born Rule Equivalence. On the Informational Limits of Real-Valued Amplitude Encodings and the Measurement of Quantum Advantage in Data Embeddings." arXiv arXiv:2602.21350.

Expressivity, trainability and stability problems: what are their **quantum** and **classical** causes?

**Hilbert state space**

(dim $\approx 2^{\text{the number of qubits}}$)
is the quantum realm where the models and their states evolve in response to unitary operations as defined by the circuit gates.

Data encoding

brings in classical data into the Hilbert space as unique and correlated quantum states during the model execution. Bad encoding prevents effective model optimisation.

Entanglements

create non-separable qubit states, which cause non-local interactions and warp the state space geometry. As a side-effect, they deform the cost landscape, affecting the optimisation process.

Circuit layers of gates

determine the evolution of the model's initial into the final state during execution (forming a state space trajectory).

Jacob L. Cybulski (2026): "A Dance of the Blind Puppeteer: The Interplay Between a Classical Optimizer and the Hilbert Space." in Zapata (Ed), A Portrait of Quantum Technologies in Finance, TQFB, 107-121.

Extras!

How to pass multiple parameters to circuits? How to create your own optimiser?

It is typical that we'd like to apply a *qnode* to *multiple inputs*. To do so, we can either apply the qnode repeatedly to consecutive rows of data, or we could run it once only by relying on the *parameter broadcasting*.

multiple-inputs

PennyLane can assign batch-size tensors to each parameter and load the qnode into a quantum device once and execute it for each set of values as needed. However, the circuit function needs to take this into consideration. For example, the *circuit1* function was written for execution in a loop, so when applied to our data, it will do exactly what we asked for – incorrect result. The function *circuit1_batched* written for batched input will work correctly.

```
@qml.qnode(dev1)
def circuit1(inputs, weights):

    # Feature map
    qml.RY(inputs[0], wires=0)
    qml.RY(inputs[1], wires=1)
    qml.Barrier()

    # Ansatz
    qml.RX(weights[0], wires=0)
    qml.RY(weights[1], wires=1)
    qml.CNOT(wires=[0, 1])
    qml.RY(weights[2], wires=1)
    qml.Barrier()

    # Measurement
    return qml.probs(wires=range(2))
```

```
@qml.qnode(dev1)
def circuit1_batched(inputs, weights):
    # Inputs are of shape (n, 2)
    # We apply RY to the feature slice
    qml.RY(inputs[:, 0], wires=0)
    qml.RY(inputs[:, 1], wires=1)
    qml.Barrier()
    # Weights are automatically broadcast
    # to match batch size
    qml.RX(weights[0], wires=0)
    qml.RY(weights[1], wires=1)
    qml.CNOT(wires=[0, 1])
    qml.RY(weights[2], wires=1)
    qml.Barrier()

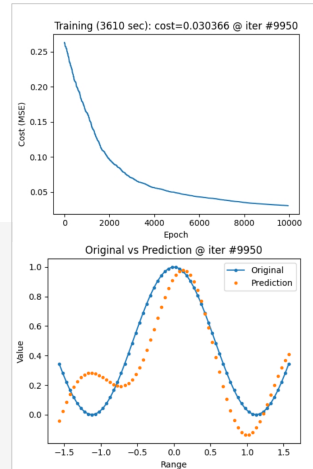
    # Measurement
    return qml.probs(wires=range(2))
```

```
[[2.19911486 0.78539816]
 [2.19911486 0.78539816]
 [2.19911486 0.78539816]
 [2.19911486 0.78539816]
 [2.19911486 0.78539816]]
```

```
class RCDOptimizer:
    def __init__(self, eta=0.01, h=0.005):
        self.h = h
        self.eta = eta

    def step(self, objective_fn, theta):
        i = np.random.randint(theta.size)
        shape = theta.shape
        dt = np.eye(1, theta.size, i)
        theta = theta.reshape(-1)
        dtheta = (objective_fn((theta+dt*self.h).reshape(shape))
                  - objective_fn((theta-dt*self.h).reshape(shape)))/(2*self.h)
        return (theta - self.eta*dtheta*dt).reshape(shape)

    def step_and_cost(self, objective_fn, theta):
        theta = self.step(objective_fn, theta)
        return theta, objective_fn(theta)
```



The mechanics of writing your own PennyLane optimiser is quite simple – all you need is to write a class implementing three methods, i.e. instance creation *init*, *step* and *step_and_cost*. The above is an example of a random coordinate descent (RCD) optimiser, which optimises a single parameter at a time – very slow, but works 😊

```
### Get the results
results4 = circuit1(multi_inputs, weights)
results4
```

```
array([[0.    , 0.502, 0.393, 0.105],
       [0.203, 0.262, 0.077, 0.458]])
```

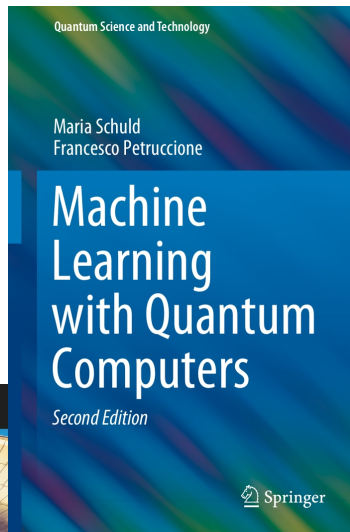
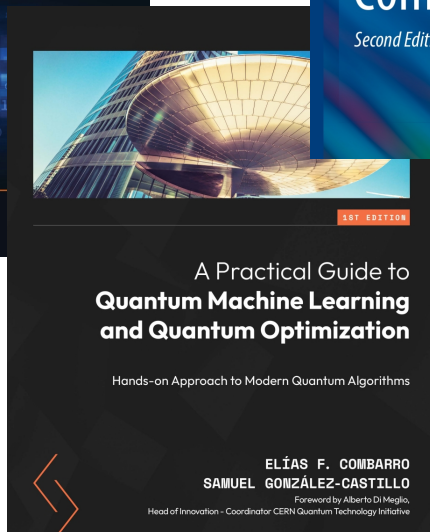
incorrect

```
### Get the results
results5 = circuit1_batched(multi_inputs, weights)
results5
```

```
array([[0.221, 0.309, 0.073, 0.397],
       [0.184, 0.346, 0.066, 0.404],
       [0.218, 0.328, 0.073, 0.381],
       [0.232, 0.309, 0.059, 0.4  ],
       [0.206, 0.333, 0.07 , 0.391]])
```

correct

Recommended reading on QML



PennyLane: Automatic differentiation of hybrid quantum-classical computations

Ville Bergholm,¹ Josh Izaac,¹ Maria Schuld,¹ Christian Gogolin,¹ M. Sohaib Alam,² Shah Nawaz Ahmed,³ Juan Miguel Arrazola,⁴ Carsten Blank,⁴ Alain Delgado,⁵ Soran Jahangiri,¹ Keri McKernan,² Johannes Jakob Meyer,⁵ Zeyu Niu,¹ Antal Száva,¹ and Nathan Killoran¹

¹Xanadu, 777 Bay Street, Toronto, Canada

²Rigetti Computing, 2019 Seventh Street, Berkeley, CA 94710

³Wallenberg Centre for Quantum Technology, Department of Microtechnology and Nanoscience, Chalmers University of Technology, 412 96 Gothenburg, Sweden

⁴Data Cybernetics, Martin-Kohnsperger-Str 26, 86899 Landsberg, Germany

⁵Dahlem Center for Complex Quantum Systems, Freie Universität Berlin, 14195 Berlin, Germany

14 Feb 2020

Modern applications of machine learning in quantum sciences

Anna Dawid^{1,2*}, Julian Arnold^{3,1}, Boris Reagena^{2,1}, Alexander Gersch^{4,1}, Marcin Płodzień^{2,1}, Kaelan Donatelli⁵, Kim A. Nicol^{6,7}, Paolo Stornati⁸, Rouven Koch⁴, Miriam Büttner⁴, Robert Okula^{9,11}, Gorka Muñoz-Gil¹², Rodrigo A. Vargas-Hernández^{13,14}, Alba Cervera-Lierta¹⁵, Juan Carrasquilla¹⁴, Vedran Dunjko¹⁶, Marylou Gabrie¹⁷, Patrick Huembeli^{18,19}, Evert van Nieuwenburg^{16,20}, Filippo Vicentini¹⁹, Lei Wang^{21,22}, Sebastian J. Wertz²³, Giuseppe Carleo¹⁸, Eliška Džepková¹, Roman Krems²⁴, Florian Marquardt^{25,27}, Michał Tomza¹, Maciej Lewenstein²⁶ and Alexandre Dauphin^{2*}

- ¹ Faculty of Physics, University of Warsaw, Poland
 - ² ICFO - Institut de Ciències Fotòniques, The Barcelona Institute of Science and Technology, 08860 Castelldefels (Barcelona), Spain
 - ³ Department of Physics, University of Basel, Switzerland
 - ⁴ Quantum Technology Research Group, Heinrich-Heine-Universität Düsseldorf, Germany
 - ⁵ Université de Paris, CNRS, Laboratoire Matière et Phénomènes Quantiques, France
 - ⁶ Machine Learning Group, Technische Universität Berlin, Germany
 - ⁷ BIFOLD, Berlin Institute for the Foundations of Learning and Data, 10587 Berlin, Germany
 - ⁸ Department of Applied Physics, Aalto University Espoo, Finland
 - ⁹ Institute of Physics, Albert-Ludwig University of Freiburg, Germany
 - ¹⁰ International Centre for Theory of Quantum Technologies, University of Gdańsk, Poland
 - ¹¹ Department of Algorithms and System Modeling, Faculty of Electronics, Faculty of Electronics, Telecommunications and Informatics, Gdańsk University of Technology, Poland
 - ¹² Institute for Theoretical Physics, University of Innsbruck, Austria
 - ¹³ Department of Chemistry, University of Toronto, Canada
 - ¹⁴ Vector Institute for Artificial Intelligence, ML6 Centre, Toronto, Canada
 - ¹⁵ Barcelona Supercomputing Center, Spain
 - ¹⁶ LIACS, Leiden University, The Netherlands
 - ¹⁷ CMAP, Ecole Polytechnique, France
 - ¹⁸ Institute of Physics, Ecole Polytechnique Fédérale de Lausanne (EPFL), Switzerland
 - ¹⁹ Menten AI, Inc., Palo Alto, California, United States of America
 - ²⁰ Niels Bohr Institute, Copenhagen, Denmark
 - ²¹ Beijing National Lab for Condensed Matter Physics and Institute of Physics, Chinese Academy of Sciences, Beijing, China
 - ²² Songshan Lake Materials Laboratory, Dongguan, China
 - ²³ Perimeter Institute for Theoretical Physics, Waterloo, Canada
 - ²⁴ Kavli Institute of Nanoscience, Delft University of Technology, NL-2600 GA Delft, The Netherlands
 - ²⁵ Department of Chemistry, University of British Columbia, Vancouver, Canada
 - ²⁶ Max Planck Institute for the Science of Light, Erlangen, Germany
 - ²⁷ Department of Physics, Friedrich-Alexander Universität Erlangen-Nürnberg, Germany
 - ²⁸ ICREA, Pg. Lluís Companys 23, 08010 Barcelona, Spain
- * These authors contributed equally.

* Anna.Dawid@fuw.edu.pl, Alexandre.Dauphin@icfo.eu

June 23, 2022

Abstract

In these Lecture Notes, we provide a comprehensive introduction to the most recent advances in the application of machine learning methods in quantum sciences. We cover the use of deep learning and kernel methods in supervised, unsupervised, and reinforcement learning algorithms for phase classification, representation of many-body quantum states, quantum feedback control, and quantum circuits optimization. Moreover, we introduce and discuss more specialized topics such as differentiable programming, generative models, statistical approach to machine learning, and quantum machine learning.



Thank you!

Any questions?



*This presentation has been released under the
Creative Commons CC BY-NC-ND license, i.e.*

BY: credit must be given to the creator.

NC: Only noncommercial uses of the work are permitted.

ND: No derivatives or adaptations of the work are permitted.

Photos from Unsplash

Enquanted is being somewhere in-between Enchanted and Entangled